Name: **Alegre, Jhon Isaac** **Bonita, Kier Nathaniel M.** **Dazon, Joshua Aareon Danielle P.** **Roldan, Serge Ulver D.**	**Date Performed: 3/19/2026**
Course/Section: CPE212/CPE31S1	**Date Submitted: 3/19/2026**
Instructor: Engr. Robin Valenzuela	**Semester and SY: Second Semester, SY 2025-2026**

Activity 11: Containerization

1. Objectives

Create a Dockerfile and form a workflow using Ansible as Infrastructure as Code (IaC) to enable Continuous Delivery process

2. Discussion

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, you can significantly reduce the delay between writing code and running it in production.

Source: https://docs.docker.com/get-started/overview/

You may also check the difference between containers and virtual machines. Click the link given below.

Source: https://docs.microsoft.com/en-us/virtualization/windowscontainers/about/containers-vs-vm

3. Tasks

1. Create a new repository for this activity.
2. Install Docker and enable the docker socket.
3. Add to Docker group to your current user.
4. Create a Dockerfile to install web and DB server.
5. Install and build the Dockerfile using Ansible.
6. Add, commit and push it to your repository.

4. Output (screenshots and explanations)

```
Setting up containerd.io (2.2.2-1~ubuntu.24.04~noble) ...
Processing triggers for man-db (2.12.0-4build2) ...
serge@serge-VirtualBox:~$ sudo systemctl start docker
serge@serge-VirtualBox:~$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/l
temd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
serge@serge-VirtualBox:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; pres
     Active: active (running) since Thu 2026-03-19 17:19:08 PST; 7min ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
```

- The image above shows Docker installed and running.

```
serge@serge-VirtualBox:~$ sudo docker ps
[sudo] password for serge:
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
serge@serge-VirtualBox:~$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/
```

- The image above is a screenshot showing that Docker's installation appears to be working correctly via the 'hello-world' command.

```
serge@serge-VirtualBox:~$ sudo usermod -aG docker $USER
serge@serge-VirtualBox:~$ newgrp docker
serge@serge-VirtualBox:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
serge@serge-VirtualBox:~$
```

- The image above is a screenshot showing a new docker group added to the user.

Dockerfile

docker_app > Dockerfile

```
# DOCKERFILE
FROM ubuntu:22.04

# PREVENT INTERACTIVE PROMPTS
ENV DEBIAN_FRONTEND=noninteractive

# UPDATE PACKAGES AND INSTALL APACHE + MYSQL
RUN apt-get update && \
    apt-get install -y apache2 mysql-server && \
    apt-get clean

# EXPOSE PORTS FOR WEB AND MYSQL
EXPOSE 80 3306

# START BOTH SERVICES
CMD service mysql start && apachectl -D FOREGROUND
```

- The image above is a screenshot of the Dockerfile that installs web and DB server.

main.yml > { } 0 > [ ] tasks > { } 0 > { } docker_image > abc source

```yaml
---
# START PLAY
- name: BUILD AND RUN DOCKER CONTAINER FROM DOCKERFILE
  hosts: localhost
  become: True
  collections:
    - community.docker
  tasks:

    - name: BUILD DOCKER IMAGE FROM DOCKERFILE
      docker_image:
        name: my-web-db
        source: build
        build:
          path: ./docker_app
        state: present

    - name: RUN DOCKER CONTAINER
      docker_container:
        name: my-web-db-container
        image: my-web-db
        state: started
```

Dockerfile main.yml

main.yml > { } 0 > [ ] tasks > { } 0 > { } docker_image > source

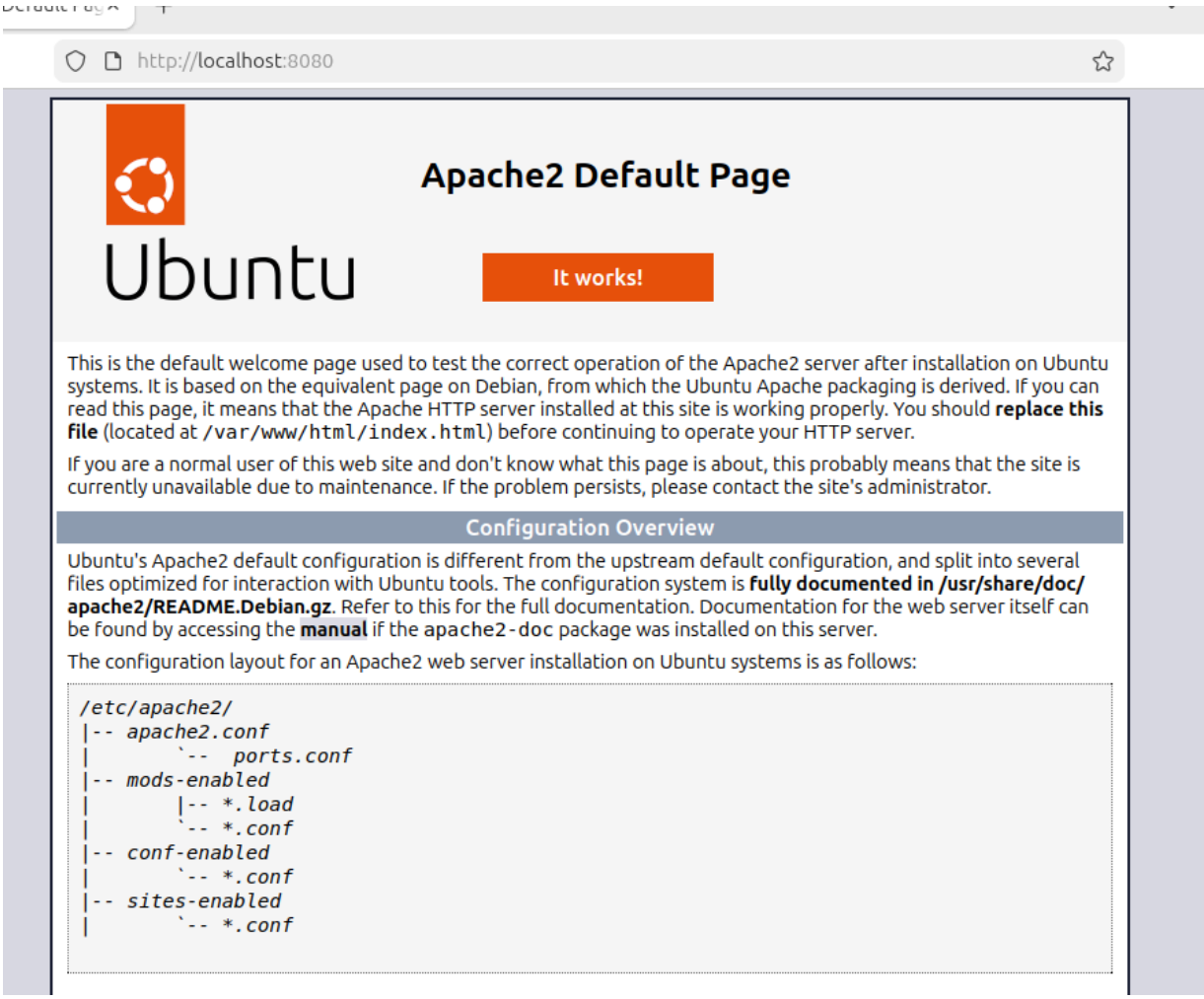
```yaml
- name: BUILD AND RUN DOCKER CONTAINER FROM DOCKERFILE
  tasks:
    - name: BUILD DOCKER IMAGE FROM DOCKERFILE
      docker_image:
        source: build
        build:
          path: ./docker_app
        state: present

    - name: RUN DOCKER CONTAINER
      docker_container:
        name: my-web-db-container
        image: my-web-db
        state: started
        ports:
          - "8080:80"
          - "3306:3306"
        restart_policy: unless-stopped
```

- The images above are screenshots of the contents of the Ansible playbook used to build and run the Dockerfile.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ANSIBLE bash

```
serge@serge-VirtualBox:~/projects/project3/docker_app$ cd ..
serge@serge-VirtualBox:~/projects/project3$ ansible-playbook main.yml -K
BECOME password:
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost d
oes not match 'all'

PLAY [BUILD AND RUN DOCKER CONTAINER FROM DOCKERFILE] ***************************************************

TASK [Gathering Facts] ***********************************************************************************
ok: [localhost]

TASK [BUILD DOCKER IMAGE FROM DOCKERFILE] ****************************************************************
changed: [localhost]

TASK [RUN DOCKER CONTAINER] ******************************************************************************
changed: [localhost]

PLAY RECAP ***********************************************************************************************
localhost                  : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0

serge@serge-VirtualBox:~/projects/project3$
```

- The image above is a screenshot of the successful build and run of the Docker container.



- The image above is a screenshot of the webpage functioning.

Reflections:

Answer the following:

1. What are the benefits of implementing containerizations?
   - It lets you run applications in a consistent way across different systems, so what works on your machine behaves the same in any system. It also starts quickly, uses fewer resources, and makes updating apps much easier without affecting others.

Conclusions:

Bonita:

In summary, containerization provides a simple and efficient way to package and run applications so they are consistent, portable, and easy to deploy across different environments; this leads to faster developments and easier management for everyone involved.

Alegre:

In this activity, our group successfully accomplished the objective of creating a Dockerfile and forming a workflow using Ansible as Infrastructure as Code (IaC) to enable a Continuous Delivery process. By installing Docker, configuring the Docker, writing a Dockerfile for web and database server deployment and automating the build process through an Ansible playbook, the group gained hands-on experience in containerization. This activity demonstrated how Docker and Ansible work together to streamline application deployment, ensuring consistency, portability, and efficiency across different environments, key principles in modern DevOps and Continuous Delivery practices.

Dazon:

For this activity, the process of creating a Dockerfile and automating its workflow using Ansible as IaC was successfully executed to establish a continuous delivery pipeline. By encapsulating the web and database servers within a container, the application is effectively separated from the underlying infrastructure, which ensures environment consistency and allows for rapid software delivery.

Conclusion (Roldan):

- I learned how Dockerfiles act as step-by-step processes to create containerized environments, and how file structure and paths are important for a build to succeed. I also learned how Ansible can automate Docker image building and container deployment, which saves time and reduces human error. Finally, I experienced the practical realities of containerization, including build time, caching, and the importance of organizing projects for reliability and repeatability.